Atividade Avaliativa — Um Novo Paradigma com NOT ONLY SQL

Projeto: EcoColeta API
Repositório: vinigoulartalves/EcoColetaAPI
Autor: Vinícius Goulart Alves
Instituição: FIAP
Disciplina: Banco de Dados NoSQL (MongoDB)
Data: Setembro/2026

---

1. Introdução e Decisão: Migração

1.1 Contexto do Projeto

O **EcoColeta API** é uma aplicação RESTful desenvolvida em ASP.NET Core 8, voltada para **gestão de resíduos e reciclagem** no contexto de **cidades inteligentes** e princípios **ESG** (Environmental, Social and Governance). A aplicação original utilizava **SQL Server** com Entity Framework Core.

1.2 Decisão: Migração (não inovação)

Optei pela **migração** do projeto existente para o MongoDB, mantendo o domínio de negócio (gestão de coleta seletiva urbana) e adaptando a estrutura relacional para o modelo documental NoSQL.

1.3 Justificativa da Abordagem NoSQL para ESG

Desafio ESG	Como o MongoDB contribui
Ambiental — Dados geoespaciais de pontos de coleta	Suporte nativo a índices `2dsphere` e documentos GeoJSON para consultas por proximidade
Ambiental — Registros de descarte com estruturas variadas	Esquema flexível: um registro pode ter `fotos`, outro `doador`, outro `evento` sem alterar o schema
Social — Orientações de descarte diversificadas	Documentos com campos opcionais (`compostagem`, `legislacao`, `destinoFinal`) por tipo de resíduo
Governança — Usuários com perfis distintos	Roles e permissões embutidas no documento (`permissoes`, `preferencias`) sem tabelas de junção
Governança — Alertas e auditoria	Histórico de alertas com metadados flexíveis (`acoesSugeridas`, `bloqueioDescarte`)
Escalabilidade — Crescimento de dados IoT/sensores	Sharding horizontal e alta disponibilidade nativos do MongoDB

---

2. As 5 Collections e Relação com ESG

2.1 `pontos_coleta` — Pilar Ambiental

Representa os pontos físicos de coleta seletiva espalhados pela cidade. Cada documento contém localização geográfica, capacidade, ocupação atual e tipo de resíduo aceito.

Exemplo de estrutura flexível: alguns pontos possuem `horarioFuncionamento`, outros `contato`, `certificacoes` ou `parceria` com ONGs.

2.2 `registros_residuos` — Pilar Ambiental

Armazena cada descarte registrado: peso, tipo, origem e vínculo com o ponto de coleta. Demonstra esquema flexível com campos opcionais como `fotos`, `classificacaoNbr`, `doador` e `evento`.

2.3 `alertas_coleta` — Pilares Ambiental + Governança

Monitora a capacidade dos pontos e gera alertas automáticos (níveis `Atencao` e `Critico`). Suporta campos extras como `acoesSugeridas`, `previsao` e `bloqueioDescarte` para governança operacional.

2.4 `destinacoes_residuos` — Pilares Ambiental + Social

Orienta cidadãos sobre o descarte correto de cada tipo de resíduo. Campos flexíveis incluem `impactoESG`, `compostagem`, `legislacao` e `destinoFinal`, educando a população (Social) e reduzindo impacto ambiental.

2.5 `usuarios_sistema` — Pilar Governança

Gerencia autenticação e autorização (Admin, Operador, Auditor, Cidadao). Esquema flexível com `preferencias`, `permissoes` e `cidadeAtuacao` conforme o perfil do usuário.

---

3. Mapeamento SQL → MongoDB

Tabela SQL (EF Core)	Collection MongoDB	Observação
`PontosColeta`	`pontos_coleta`	`Latitude/Longitude` → `localizacao` GeoJSON
`RegistrosResiduos`	`registros_residuos`	`PontoColetaId` (int) → `pontoColetaId` (ObjectId)
`AlertasColeta`	`alertas_coleta`	Relacionamento por referência (ObjectId)
`DestinacoesResiduos`	`destinacoes_residuos`	Campos ESG adicionais no documento
`UsuariosSistema`	`usuarios_sistema`	Permissões embutidas no documento

---

4. Configuração do Ambiente MongoDB

4.1 Via Docker Compose (recomendado)

Na raiz do projeto:

```
docker compose up mongodb -d
```

O serviço MongoDB sobe na porta 27017 com o banco `EcoColetaDb`.

4.2 Executar script de seed e CRUD

```
# Com MongoDB rodando localmente
MONGODB_URI=mongodb://localhost:27017 node mongodb/scripts/run-evidencias.js

# Ou via mongosh
mongosh EcoColetaDb < mongodb/ecocoleta-mongodb-crud.js
```

4.3 Arquivos do projeto MongoDB

Arquivo	Descrição
`mongodb/ecocoleta-mongodb-crud.js`	Script completo para mongosh (seed + CRUD)
`mongodb/scripts/seed-data.js`	Dados de seed (11+ docs por collection)
`mongodb/scripts/run-evidencias.js`	Executa seed + CRUD e gera evidências
`mongodb/evidencias/crud-evidencias.txt`	Saída das operações (evidência textual)

---

5. Comandos de Criação das Collections

No MongoDB, collections são criadas implicitamente no primeiro `insert`. Os índices são criados explicitamente:

```
use EcoColetaDb

// Índices - pontos_coleta
db.pontos_coleta.createIndex({ cidade: 1, bairro: 1 })
db.pontos_coleta.createIndex({ localizacao: "2dsphere" })
db.pontos_coleta.createIndex({ tipoResiduoAceito: 1 })

// Índices - destinacoes_residuos
db.destinacoes_residuos.createIndex({ tipoResiduo: 1 })

// Índices - usuarios_sistema
db.usuarios_sistema.createIndex({ email: 1 }, { unique: true })

// Índices - registros_residuos
db.registros_residuos.createIndex({ pontoColetaId: 1, registradoEm: -1 })

// Índices - alertas_coleta
db.alertas_coleta.createIndex({ resolvido: 1, nivel: 1 })
```

---

6. Operações CRUD — Todas as Collections

6.1 CREATE (Insert)

pontos_coleta (11 documentos)

```
db.pontos_coleta.insertMany([
  {
    nome: "EcoPonto Centro",
    endereco: "Rua das Flores, 100",
    bairro: "Centro",
    cidade: "São Paulo",
    localizacao: { type: "Point", coordinates: [-46.6333, -23.5505] },
    tipoResiduoAceito: "Plastico",
    capacidadeMaximaKg: 500,
    ocupacaoAtualKg: 120,
    ativo: true,
    horarioFuncionamento: { abertura: "08:00", fechamento: "18:00", diasUteis: true }
  },
  {
    nome: "EcoPonto Jardins",
    endereco: "Av. Paulista, 500",
    bairro: "Jardins",
    cidade: "São Paulo",
    localizacao: { type: "Point", coordinates: [-46.6559, -23.5614] },
    tipoResiduoAceito: "Papel",
    capacidadeMaximaKg: 300,
    ocupacaoAtualKg: 85,
    ativo: true,
    contato: { telefone: "(11) 3000-1000", email: "jardins@ecocoleta.com.br" }
  }
  // ... mais 9 documentos (ver mongodb/ecocoleta-mongodb-crud.js)
])
```

destinacoes_residuos (11 documentos)

```
db.destinacoes_residuos.insertMany([
  {
    tipoResiduo: "Plastico",
    descricao: "Plásticos recicláveis",
    instrucoesDescarte: "Lavar e secar antes do descarte.",
    reciclavel: true,
    riscoAmbiental: "Baixo quando reciclado",
    impactoESG: { pilar: "Ambiental", reducaoCO2KgPorTonelada: 1500 }
  },
  {
    tipoResiduo: "Eletronico",
```

```
    descricao: "Equipamentos eletrônicos",
    instrucoesDescarte: "Levar a pontos especializados.",
    reciclavel: true,
    riscoAmbiental: "Alto - metais pesados",
    legislacao: { lei: "PNRS", artigo: "Logística reversa" }
  }
  // ... mais 9 documentos
])
```

usuarios_sistema (11 documentos)

```
db.usuarios_sistema.insertMany([
  { nome: "Administrador", email: "admin@ecocoleta.com", senhaHash: "$2a$11$hash", role: "Admin
  { nome: "Ana Costa", email: "ana.costa@ecocoleta.com", senhaHash: "$2a$11$hash", role: "Audit
  // ... mais 9 documentos
])
```

registros_residuos (12 documentos)

```
db.registros_residuos.insertMany([
  { pontoColetaId: ObjectId("..."), pontoColetaNome: "EcoPonto Centro", tipoResiduo: "Plastico"
  { pontoColetaId: ObjectId("..."), pontoColetaNome: "EcoPonto Vila Mariana", tipoResiduo: "Vid
  { pontoColetaId: ObjectId("..."), pontoColetaNome: "EcoPonto Guarulhos", tipoResiduo: "Plasti
  // ... mais 9 documentos
])
```

alertas_coleta (11 documentos)

```
db.alertas_coleta.insertMany([
  { pontoColetaNome: "EcoPonto Vila Mariana", nivel: "Atencao", mensagem: "Ocupação atingiu 80%
  { pontoColetaNome: "EcoPonto Guarulhos", nivel: "Critico", mensagem: "Ponto sobrecarregado",
  // ... mais 9 documentos
])
```

6.2 READ (Find / Aggregate)

```
// Contar documentos
db.pontos_coleta.countDocuments()  // => 11

// Buscar pontos ativos em São Paulo
db.pontos_coleta.find({ cidade: "São Paulo", ativo: true }).limit(3)

// Destinações de alto risco ambiental
db.destinacoes_residuos.find({ reciclavel: true, riscoAmbiental: /Alto/ })

// Alertas críticos não resolvidos
db.alertas_coleta.find({ resolvido: false, nivel: "Critico" })

// Agregação: total de kg por tipo de resíduo
```

```
db.registros_residuos.aggregate([
  { $group: { _id: "$tipoResiduo", totalKg: { $sum: "$pesoKg" }, quantidade: { $sum: 1 } } },
  { $sort: { totalKg: -1 } }
])

// Usuários auditores com permissão de governança
db.usuarios_sistema.find({ role: "Auditor", permissoes: "governanca" })
```

6.3 UPDATE

```
// Atualizar ocupação de um ponto
db.pontos_coleta.updateOne(
  { nome: "EcoPonto Centro" },
  { $set: { ocupacaoAtualKg: 135, ultimaAtualizacao: new Date() } }
)

// Resolver alerta
db.alertas_coleta.updateOne(
  { pontoColetaNome: "EcoPonto Santana", resolvido: false },
  { $set: { resolvido: true, resolvidoEm: new Date() } }
)

// Atualizar preferência de usuário (campo aninhado)
db.usuarios_sistema.updateOne(
  { email: "maria.silva@ecocoleta.com" },
  { $set: { "preferencias.tema": "escuro" } }
)

// Incrementar impacto ESG de plásticos
db.destinacoes_residuos.updateMany(
  { tipoResiduo: "Plastico" },
  { $inc: { "impactoESG.reducaoCO2KgPorTonelada": 50 } }
)

// Atualizar peso de registro
db.registros_residuos.updateOne(
  { pontoColetaNome: "EcoPonto Centro", tipoResiduo: "Plastico" },
  { $set: { observacao: "Reclassificado após auditoria" } }
)
```

6.4 DELETE

```
// Remover ponto inativo
db.pontos_coleta.deleteOne({ nome: "EcoPonto Inativo Teste" })

// Remover usuário desativado
db.usuarios_sistema.deleteOne({ email: "inativo@ecocoleta.com" })
```

```
// Remover alertas resolvidos antigos (exemplo)
db.alertas_coleta.deleteMany({ resolvido: true, resolvidoEm: { $lt: new Date("2026-01-01") } })
```

---

7. Esquema Flexível — Evidência

Documentos na mesma collection `pontos_coleta` com estruturas diferentes:

```
// Documento com horário de funcionamento
{ "nome": "EcoPonto Centro", "horarioFuncionamento": { "abertura": "08:00", "fechamento": "18:0

// Documento com contato
{ "nome": "EcoPonto Jardins", "contato": { "telefone": "(11) 3000-1000", "email": "jardins@ecoc

// Documento com certificações ambientais
{ "nome": "EcoPonto Pinheiros", "certificacoes": ["ISO 14001", "Selo Verde SP"] }

// Documento com parceria social
{ "nome": "EcoPonto Mooca", "parceria": { "ong": "Composta Cidade", "tipo": "compostagem comuni
```

Isso demonstra a **flexibilidade de esquema** do MongoDB, impossível em tabelas SQL sem colunas nullable ou tabelas auxiliares.

---

8. Evidências das Operações

A execução do script `mongodb/scripts/run-evidencias.js` gerou o arquivo `mongodb/evidencias/crud-evidenci` com a saída completa de todas as operações.

Resumo após CRUD completo

Collection	Documentos
`pontos_coleta`	11
`destinacoes_residuos`	11
`usuarios_sistema`	10
`registros_residuos`	12
`alertas_coleta`	11

Prints de tela

> **Instrução para o aluno:** execute os comandos abaixo no seu ambiente local com MongoDB Compass ou mongosh aberto, e capture prints de tela de cada operação (insert, find, update, delete). Insira as imagens neste documento antes de converter para PDF.

Comandos sugeridos para captura de prints:

1. `show collections` — listar as 5 collections
2. `db.pontos_coleta.find().limit(3).pretty()` — evidência de INSERT/READ
3. `db.alertas_coleta.find({ resolvido: false })` — evidência de READ com filtro

4. `db.pontos_coleta.updateOne(...)` seguido de `findOne(...)` — evidência de UPDATE
5. `db.pontos_coleta.deleteOne(...)` seguido de `countDocuments()` — evidência de DELETE

---

9. Conclusão

A migração do EcoColeta API para MongoDB demonstra como bancos NoSQL atendem desafios ESG em cidades inteligentes:

- **Flexibilidade** para dados heterogêneos (pontos com certificações, registros com doadores, alertas com previsões).
- **Escalabilidade** para crescimento de sensores IoT e registros de cidadãos.
- **Consultas geoespaciais** nativas para localizar pontos de coleta próximos.
- **Agregações** eficientes para relatórios de impacto ambiental.
- **Governança** com permissões embutidas nos documentos de usuário.

O projeto mantém a API REST original (SQL Server) e adiciona a camada MongoDB como demonstração do paradigma NoSQL, atendendo todos os requisitos da atividade avaliativa.

---

10. Referências

- MongoDB Documentation — https://www.mongodb.com/docs/
- FIAP — Atividade "Um novo paradigma com NOT ONLY SQL"
- Repositório EcoColeta API — https://github.com/vinigoulartalves/EcoColetaAPI

Anexo opcional: `mongodb/ecocoleta-mongodb-crud.js` — script completo de criação e manipulação de dados.